

SKALA

AI-Agent Capstone(+MCP)

2026.10

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX



목차

1. MCP 기본 구조
2. 도구 네 개로 늘리기
3. Resource와 Prompt
4. MCP 통신 방식
5. 레퍼런스 아키텍처
6. Harness



1. MCP 기본 구조

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

MCP를 구성하는 세 가지 요소

MCP Server는 외부에 제공할 기능(Tool)을 정의하고, **MCP Client**는 Server에 연결해 그 기능을 찾아 호출한다. 둘 사이에서 기능을 발견하고 호출하는 표준 통신 규약이 **MCP Protocol**이다.

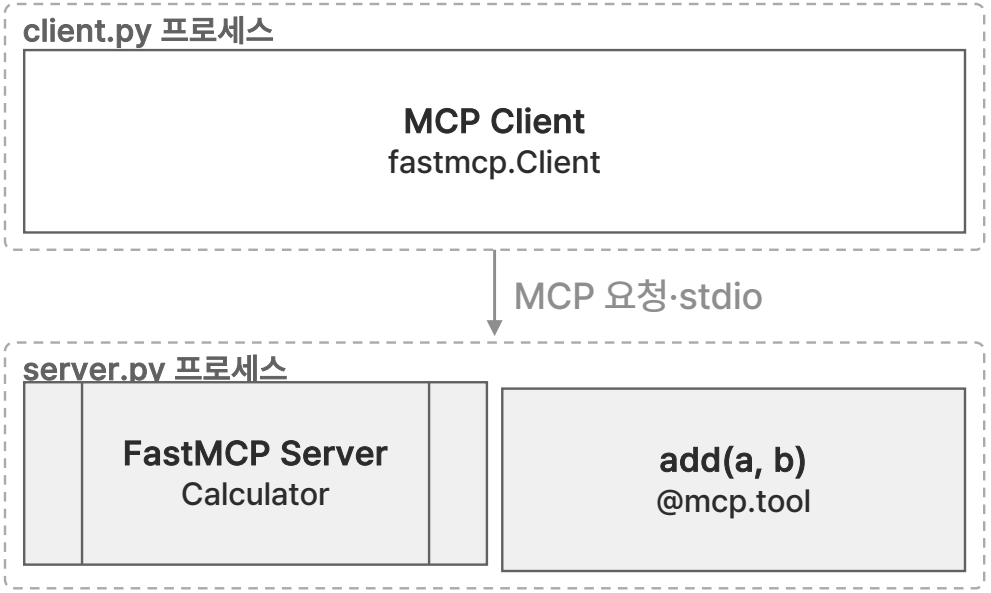


그림 1-1 계산기 예제의 Client-Server 구조

표 1-1 세 구성 요소

구성 요소	계산기 예제에서
MCP Server	add(a, b) 도구 하나를 가진 server.py
MCP Client	server.py를 실행하고 add를 호출하는 client.py
MCP Protocol	list_tools()로 찾고 call_tool()로 호출한다. 메시지는 JSON-RPC이다

함수 호출과 MCP 호출의 차이

일반 파이썬 코드는 함수를 import해 바로 호출한다. MCP Client는 Server의 함수를 import하지 않고, 표준 규약으로 Tool을 발견하고 호출한다. 함수는 같고, 함수를 외부에 공개하고 호출하는 방식이 다르다.

표 1-2 파이썬 import와 MCP 호출

항목	파이썬 import	MCP
호출하는 코드	from calculator import add add(10, 20)	await client.call_tool("add", {"a": 10, "b": 20})
함수를 아는 방법	소스 코드를 읽는다	list_tools()로 이름, 설명, 인자 스키마를 받는다
실행 위치	호출한 쪽과 같은 프로세스	Server 프로세스 (stdio 또는 HTTP)
언어	같아야 한다	달라도 된다. 규약만 맞으면 Java Client도 호출한다
무엇을 호출할지 정하는 쪽	개발자	1·2단계는 개발자, 3단계는 모델

MCP 구성 요소별 목적과 사용 시점

계산기 예제 하나에 Host, MCP Client, MCP Server, Tool, 전송, JSON-RPC 메시지가 모두 등장한다. 요소마다 존재 이유와 사용하는 시점이 다르므로, 코드를 읽을 때 어느 요소의 코드인지 먼저 구분한다.

표 1-3 MCP 구성 요소의 목적과 용도

요소	목적	언제 사용하나	계산기 예제에서
Host 애플리케이션	사용자와 LLM 사이에서 MCP Client를 실행하는 프로그램	LLM이 외부 기능을 사용해야 할 때. LLM이 없는 1·2단계는 Client 코드가 이 위치에 있다	3단계 agent.py (1·2단계는 client.py)
MCP Client	Server 하나와 연결을 유지하고 목록·호출 요청을 보낸다	도구 목록을 받을 때(list_tools), 도구를 실행할 때(call_tool)	fastmcp.Client(SERVER)
MCP Server	기능을 Tool·Resource·Prompt로 공개하고 요청을 실행한다	같은 기능을 여러 Client, 다른 언어, 모델이 함께 사용할 때	FastMCP("Calculator")
Tool	Server가 실행하는 함수 하나. 이름·설명·인자 스키마를 함께 공개한다	계산, 조회, 변경처럼 실행 결과가 필요한 일	add(a, b)
전송 (Transport)	메시지를 전달하는 경로. 업무 코드와 분리된다	로컬 개발은 stdio, 따로 실행한 Server는 Streamable HTTP	stdio. Client가 Server를 하위 프로세스로 실행한다
JSON-RPC 메시지	요청과 응답의 형식. id로 요청과 응답을 매칭한다	모든 요청 (server/discover, tools/list, tools/call)	stdio에서는 한 줄에 JSON 하나

MCP가 동작하는 일곱 단계

Client의 async with가 연결을, list_tools()가 목록 요청을, call_tool()이 실행 요청을 보낸다. 1단계에서는 무엇을 호출할지도 아직 Client 코드가 정한다.

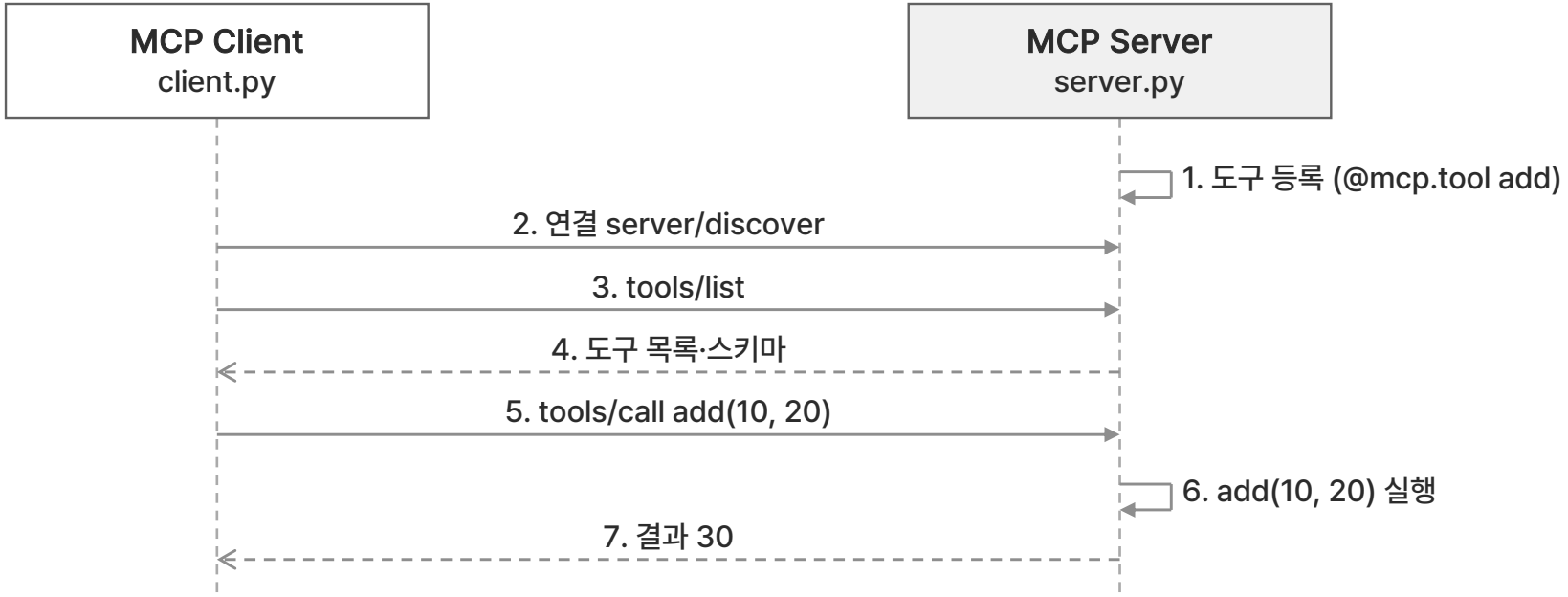


그림 1-2 Client와 Server가 주고받는 순서 (Sequence)

> **알아두기**
> FastMCP 4.1이 사용하는 프로토콜 2026-07-28은 연결할 때 server/discover로 서로의 버전과 기능을 확인한다. 이전 명세에서는 같은 단계에서 initialize 요청과 initialized 알림이 있었다.

AI Agent 연결 구조

계산기 예제에는 LLM이 없어 Client 코드가 직접 add를 선택한다. AI Agent를 더하면 LLM이 사용할 Tool을 판단하고, Agent가 그 판단에 따라 실행 흐름을 제어한다. MCP는 AI Agent 자체가 아니라, Agent가 외부 기능을 표준화된 방식으로 사용하도록 돕는 프로토콜이다.

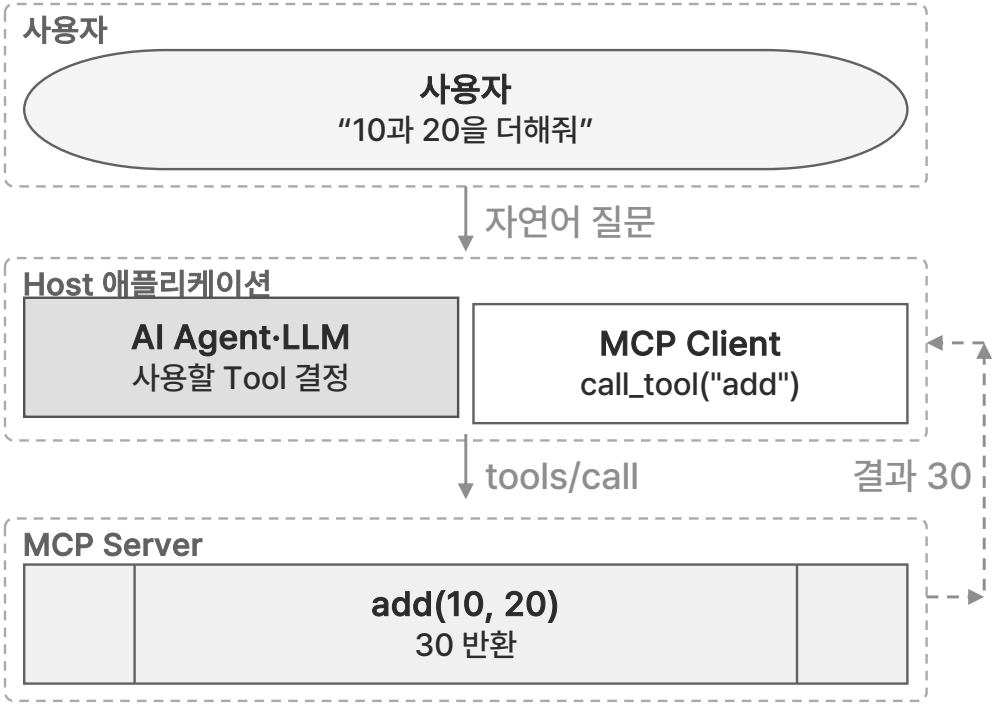


그림 1-3 Agent를 더한 구조 (Client-Server)

표 1-4 네 가지 역할

구성 요소	역할
LLM	어떤 Tool을 사용할지 판단한다
AI Agent	LLM 판단에 따라 실행 흐름을 제어한다
MCP Client	MCP Server의 Tool을 호출한다
MCP Server	실제 기능을 실행한다

계산기 Server 만들기

@mcp.tool 데코레이터가 일반 파이썬 함수를 Client가 호출할 수 있는 Tool로 등록한다. 함수 이름은 도구 이름, 인자 타입은 인자 스키마, docstring은 도구 설명이 된다.

예제 1-1 server.py

```
from fastmcp import FastMCP          # MCP Server를 만드는 클래스

# 1. Server 객체를 만든다. 인자 "Calculator"는 Server 이름이다
mcp = FastMCP("Calculator")

# 2. @mcp.tool 데코레이터가 일반 파이썬 함수를 MCP 도구로 등록한다
#    함수 이름 → name, 인자 타입 → inputSchema, 반환 타입 → outputSchema
#    docstring → description. 모델이 도구를 선택할 때 읽는다
@mcp.tool
def add(a: int, b: int) -> int:
    """두 숫자를 더한다."""
    return a + b                     # 반환값은 structuredContent의 result로 전달된다

# 3. 직접 실행할 때만 동작한다. 인자 없는 run()은 stdio로 요청을 기다린다
if __name__ == "__main__":
    mcp.run(show_banner=False)       # 시작할 때 출력하는 배너를 표시하지 않는다
```

> **알아두기**
> 이 파일은 직접 실행하지 않아도 된다. client.py가 이 파일을 하위 프로세스로 실행하고, 표준 입출력으로 연결한다.

@mcp.tool이 만드는 도구 정의

Client가 도구 목록을 요청하면 Server는 아래 정의를 돌려준다. 1단계 Client를 stdio 중계로 감싸 실제 응답을 기록했다. 모델도 함수 본문은 보지 않고 이 정의만 읽어 도구를 선택한다.

예제 1-2 tools/list 응답에 실린 add 도구 (FastMCP 4.1, _meta 생략)

```
{
  "name": "add",                          # 함수 이름
  "title": "Add",                         # 이름에서 자동으로 만든 표시용 제목
  "description": "두 숫자를 더한다.",      # docstring
  "inputSchema": {                        # 인자 타입 힌트 → JSON Schema
    "type": "object",
    "properties": {"a": {"type": "integer"}, "b": {"type": "integer"}},
    "required": ["a", "b"],               # 기본값이 없는 인자
    "additionalProperties": false
  },
  "outputSchema": {                       # 반환 타입 int → {"result": int}로 감싼다
    "type": "object",
    "properties": {"result": {"type": "integer"}},
    "required": ["result"]
  }
}
```

> **알아두기**
> 반환 타입이 int처럼 객체가 아니면 FastMCP가 {"result": ...}로 감싸 outputSchema를 만든다. Client는 이 정의를 보고 result.data로 int 30을 돌려준다.

계산기 Client 만들기

Client에 .py 경로를 넘기면 Client가 Server를 stdio 방식으로 직접 실행하고 연결하므로, Server 터미널을 따로 열 필요가 없다.

예제 1-3 client.py

```
import asyncio                          # async 함수를 실행하는 표준 라이브러리
from pathlib import Path                # 파일 경로를 OS와 관계없이 다루는 표준 라이브러리
from fastmcp import Client              # MCP Client

# 이 파일(client.py)과 같은 폴더에 있는 server.py의 경로
#   __file__           지금 실행 중인 파일의 경로   …/client.py
#   Path(…).parent     그 파일이 있는 폴더          …/ (client.py가 있는 폴더)
#   / "server.py"      폴더 경로 뒤에 파일 이름을 이은 새 경로
# 터미널의 현재 위치와 관계없이 같은 폴더의 server.py를 찾는다
# 문자열이 아니라 Path로 넘긴다 (문자열 경로는 FastMCP 5에서 제거될 예정이다)
SERVER = Path(__file__).parent / "server.py"

async def main():
    # 1. .py 경로를 넘기면 Client가 server.py를 하위 프로세스로 실행하고
    #    표준 입출력(stdio)으로 연결한다. Server 터미널을 따로 열지 않는다
    client = Client(SERVER)

    # 2. async with에 들어갈 때 연결하고(server/discover),
    #    블록을 나올 때 연결을 닫고 Server 프로세스를 종료한다
    async with client:
```

계산기 Client 만들기 (계속)

예제 1-3 client.py (계속)

```
        # 3. tools/list 요청. 이름·설명·인자 스키마가 담긴 도구 목록을 받는다
        tools = await client.list_tools()
        for tool in tools:
            print("Tool:", tool.name)

        # 4. tools/call 요청. 도구 이름과 인자(dict)를 넘긴다
        result = await client.call_tool("add", {"a": 10, "b": 20})
        # result.data는 반환 타입(int)으로 변환한 값이다
        # 원본 JSON은 result.structured_content ({"result": 30})
        print("결과:", result.data)

# main()은 코루틴이므로 asyncio.run()이 이벤트 루프를 만들어 실행한다
asyncio.run(main())

# 실행 결과
# Tool: add
# 결과: 30
```

설치하고 실행하기

실습 환경 준비에서 만든 가상환경을 켜고 calculator 폴더에서 client.py를 실행한다.

예제 1-4 계산기 실행 (macOS·Linux)

```
cd 전체코드
source .venv/bin/activate              # Windows는 .venv\Scripts\activate
cd calculator
python client.py                       # Tool: add / 결과: 30
```

> **알아두기**
> server.py는 직접 실행하지 않는다. client.py가 자기 파일 위치를 기준으로 같은 폴더의 server.py를 찾아 stdio로 실행한다.

실제로 주고받은 JSON-RPC 메시지

MCP 메시지는 JSON-RPC 2.0 형식이다. 요청에는 id와 method가 있고, 응답은 같은 id로 돌아온다. stdio에서는 JSON 하나가 한 줄이다.

예제 1-5 1단계 실행 중 stdio 기록 (_meta 생략)

```
# ① 연결 (async with)
C→S {"jsonrpc":"2.0","id":1,"method":"server/discover"}
S→C {"jsonrpc":"2.0","id":1,"result":{"supportedVersions":["2026-07-28"],
      "capabilities":{"tools":{},"resources":{},"prompts":{},"logging":{}}}}

# ② 도구 목록 (list_tools)
C→S {"jsonrpc":"2.0","id":2,"method":"tools/list"}
S→C {"jsonrpc":"2.0","id":2,"result":{"tools":[{"name":"add",
      "description":"두 숫자를 더한다.","inputSchema":{...},"outputSchema":{...}}]}}

# ③ 도구 호출 (call_tool("add", {"a": 10, "b": 20}))
C→S {"jsonrpc":"2.0","id":3,"method":"tools/call",
      "params":{"name":"add","arguments":{"a":10,"b":20}}}
S→C {"jsonrpc":"2.0","id":3,"result":{"content":[{"type":"text","text":"30"}],
      "structuredContent":{"result":30},"isError":false}}
```

> **알아두기**
> 결과는 사람이 읽는 content와 프로그램이 읽는 structuredContent로 함께 반환된다. isError가 true이면 도구 실행이 실패했다는 뜻이다. FastMCP Client는 이때 ToolError를 낸다.

1단계 따라하기 점검표

계산기 Server와 Client를 직접 실행하며 순서대로 확인한다. 결과가 표와 다르면 오른쪽 열부터 본다.

표 1-5 1단계 실행 순서

순서	할 일 (명령)	보여야 하는 결과	다르면 확인할 것
1	cd 전체코드 후 source .venv/bin/activate	프롬프트 앞에 (.venv)가 보인다	ModuleNotFoundError가 발생하면 가상환경이 꺼져 있다
2	cd calculator 후 server.py를 편집기로 연다	server.py에 @mcp.tool이 붙은 add 함수가 있다	폴더 이름을 확인한다 (calculator)
3	python client.py	Tool: add, 결과: 30	INFO 로그 줄은 정상이다. 오류면 마지막 줄을 읽는다
4	client.py의 {"a": 10, "b": 20}을 {"a": 7, "b": 5}로 바꿔 저장하고 다시 실행	결과: 12	저장했는지 확인한다

MCP 기본 구조 Summary

표 1-6 MCP 기본 구조 요점

구분	요점
구성 요소	MCP Server가 Tool을 정의하고, MCP Client가 Server에 연결해 Tool을 찾아 호출한다. 둘 사이의 통신 규약이 MCP Protocol이다
함수 호출과 차이	함수를 import하지 않고, 표준 규약으로 Tool을 발견하고 호출한다
동작 순서	연결(server/discover) → tools/list → tools/call. 메시지는 JSON-RPC 2.0이다
예제 코드	@mcp.tool 데코레이터가 함수 이름·인자 타입·docstring으로 도구 정의를 만든다. Client(SERVER)는 server.py를 stdio로 실행한다
AI Agent	LLM이 사용할 Tool을 판단하고 Agent가 실행 흐름을 제어한다. MCP는 그 사이의 프로토콜이다



2. 도구 네 개로 늘리기

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

도구 정의 요소와 쓰임

계산기 Server의 도구는 add 하나였다. 도구가 네 개로 늘면 Client와 모델이 도구를 구분할 정보가 필요하고, 그 정보는 모두 함수 정의에서 만들어진다.

표 2-1 함수 정의가 도구 정의가 되는 방식

함수의 요소	도구 정의에서	목적	상품 조회 Server에서
함수 이름	name	호출할 도구를 가리킨다	get_price, get_stock, multiply, add
docstring	description	모델이 도구를 선택할 때 읽는다. 받는 값과 돌려주는 값, 단위까지 적는다	상품 이름(예: 사과)을 받아 한 개 가격(원)을 돌려준다.
인자 타입 힌트	inputSchema	인자 이름·타입·필수 여부를 Client와 모델에 전달한다	product: str
반환 타입	outputSchema, structuredContent	결과를 형식이 있는 값으로 받게 한다	int → {"result": int}
ToolError	isError: true와 메시지	실패 이유와 다시 시도할 때 필요한 정보를 전달한다	없는 상품이면 있는 상품 목록을 메시지에 넣는다
@mcp.tool이 없는 함수	공개되지 않는다	공통 로직을 도구 목록에서 감춘다	find_product()

실습 구성과 실행(참조만)

계산기 Server를 확장해 상품 가격·재고 조회와 계산 도구 네 개를 가진 Server를 만든다. 사용자가 “사과 3개의 가격은?”이라고 물으면 get_price(“사과”)와 multiply(1500, 3)을 차례로 호출해 4500을 얻는다. 2단계에서는 그 순서를 코드가 정한다.

예제 2-1 단계별 파일 구성과 실행

```
calculator/  server.py  client.py    # 1단계. add 도구 하나
shop/        server.py  client.py    # 2단계. 도구 네 개, 코드가 호출 순서를 정한다
shop/        agent.py                # 3단계. 모델이 호출 순서를 정한다 (create_agent)

cd shop
python client.py                     # 2단계. 사과 3개 가격: 4500
python agent.py                      # 3단계. OPENAI_API_KEY 환경 변수가 필요하다
```

> **알아두기**
> 이번 실습은 Python, FastMCP, LangChain, LangGraph만 사용한다. Spring Boot와 Vue 3는 MCP 호출 구조를 익힌 뒤에 연결한다.

상품 조회 Server

get_price와 get_stock은 같은 find_product()를 거치고, 없는 상품이면 ToolError를 발생시켜 그 메시지를 Client와 모델에게 그대로 전달한다.

예제 2-2 server.py (발췌)

```
from fastmcp import FastMCP
from fastmcp.exceptions import ToolError    # 도구 실패를 Client에 전달하는 예외
mcp = FastMCP("ShoppingServer")

# 실습용 데이터. 키는 상품 이름, 값은 가격(원)과 재고(개)
# Level 3에서 이 dict를 PostgreSQL 조회로 바꾼다. 도구 정의는 그대로 둔다
PRODUCTS = {"사과": {"price": 1500, "stock": 100}, "바나나": {"price": 2000, "stock": 50},
            "오렌지": {"price": 3000, "stock": 30}}

# get_price와 get_stock이 함께 사용하는 조회 함수
# @mcp.tool이 없으므로 도구로 공개되지 않는다
def find_product(product: str) -> dict:
    item = PRODUCTS.get(product)            # 없는 상품이면 None
    if item is None:
```

상품 조회 Server (계속)

예제 2-2 server.py (발췌) (계속)

```
        # ToolError의 메시지는 Client와 모델에게 그대로 전달된다 (isError: true)
        # 재시도에 필요한 정보(있는 상품 목록)를 메시지에 넣는다
        raise ToolError(f"'{product}'은(는) 없는 상품이다. 있는 상품: 사과, 바나나, 오렌지")
    return item

# docstring이 도구 설명이 된다. 모델은 이 설명을 읽고 도구를 선택하므로
# 무엇을 받아 무엇을 돌려주는지 단위까지 적는다
@mcp.tool
def get_price(product: str) -> int:
    """상품 이름(예: 사과)을 받아 한 개 가격(원)을 돌려준다."""
    return find_product(product)["price"]
# get_stock(product) 재고, multiply(a, b) 곱셈, add(a, b) 덧셈도 같은 형태이다
```

> **알아두기**
>
> 오류 메시지에 “있는 상품: …”처럼 재시도에 필요한 정보를 넣는다. 3단계 모델은 이 정보로 다시 호출한다.

코드가 호출 순서를 정하는 Client

이 단계에는 LLM이 없다. get_price의 결과 1500을 price 변수에 저장해 multiply의 인자로 넘기는 코드는 개발자가 작성했다. 3단계에서는 이 연결을 모델이 질문을 읽고 정한다.

예제 2-3 client.py (발췌)

```
async def main():
    # SERVER는 1단계와 같은 방식의 경로이다. async with가 연결과 종료를 담당한다
    async with Client(SERVER) as client:
        # 1. 도구 목록과 설명을 출력한다. 3단계에서 모델이 읽는 설명이 이것이다
        tools = await client.list_tools()
        for tool in tools:
            print(tool.name, "-", tool.description)

        # 2. 사과 한 개 가격. result.data는 int 1500이다
        result = await client.call_tool("get_price", {"product": "사과"})
        price = result.data

        # 3. 앞 결과(price)를 다음 인자로 넘긴다. 순서와 인자를 코드가 정한다
        result = await client.call_tool("multiply", {"a": price, "b": 3})
        print("사과 3개 가격:", result.data)

        # 4. 없는 상품은 ToolError 예외로 발생한다 (기본값 raise_on_error=True)
        try:
            await client.call_tool("get_price", {"product": "포도"})
        except ToolError as error:
            print("오류:", error)          # 오류: '포도'은(는) 없는 상품이다. …
```

2단계 실행 결과

도구 목록, 사과 가격과 3개 가격, 없는 상품의 오류 메시지가 차례로 출력된다.

예제 2-4 python client.py 실행 결과

```
=== MCP Tool 목록 ===
get_price - 상품 이름(예: 사과)을 받아 한 개 가격(원)을 돌려준다.
get_stock - 상품 이름을 받아 현재 재고 수량(개)을 돌려준다.
multiply - 두 정수를 곱한다. 가격 × 수량 계산에 쓴다.
add - 두 정수를 더한다. 여러 상품 금액을 합칠 때 쓴다.

사과 가격: 1500
사과 3개 가격: 4500
오류: '포도'은(는) 없는 상품이다. 있는 상품: 사과, 바나나, 오렌지
```

> **알아두기**
> 이 단계까지는 LLM이 개입하지 않고, 호출할 Tool과 매개변수를 개발자가 직접 지정한다.
> 3단계에서는 같은 도구 목록과 설명을 모델에게 넘기고, 모델이 질문을 읽어 get_price와 multiply를 선택한다.

2단계 직접 실행과 확인

상품 조회 Server와 Client를 실행해 도구 목록, 두 도구의 연결, 오류 전달을 출력으로 확인한다.

표 2-2 2단계 실행 순서

순서	할 일 (명령)	보여야 하는 결과	다르면 확인할 것
1	shop 폴더에서 python client.py	도구 네 개의 이름과 설명이 출력된다	도구 설명은 server.py의 docstring과 같은 문장이다
2	1번 출력의 나머지 줄	사과 가격: 1500, 사과 3개 가격: 4500	가격은 server.py의 PRODUCTS에서 온다
3	1번 출력의 마지막 줄	오류: '포도'은(는) 없는 상품이다. 있는 상품: 사과, 바나나, 오렌지	Error calling tool 'get_price' 로 그는 일부러 낸 오류라 정상이다
4	server.py에서 get_price의 docstring을 찾는다	도구 목록 출력과 같은 문장이다	모델은 이 문장만 읽고 도구를 선택한다

도구 네 개로 늘리기 Summary

표 2-3 도구 네 개로 늘리기 요점

구분	요점
Server	get_price, get_stock, multiply, add 네 도구를 가진 상품 조회 Server
오류 처리	없는 상품은 ToolError로 반환하고, 메시지에 재시도에 필요한 정보(있는 상품 목록)를 넣는다
호출 순서	2단계는 호출 순서와 인자를 개발자가 코드로 정한다. 3단계에서는 모델이 정한다
docstring	도구 목록에 출력되는 설명이며, 3단계에서 모델이 도구를 선택할 때 읽는다



3. Resource와 Prompt

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

MCP Server가 제공하는 세 가지 기능

1·2단계의 Server는 Tool만 제공했다. MCP Server는 Tool 외에도 읽기 전용 자료인 Resource와 미리 만들어 둔 지시문인 Prompt를 제공하고, 셋은 누가 선택하느냐로 구분한다.

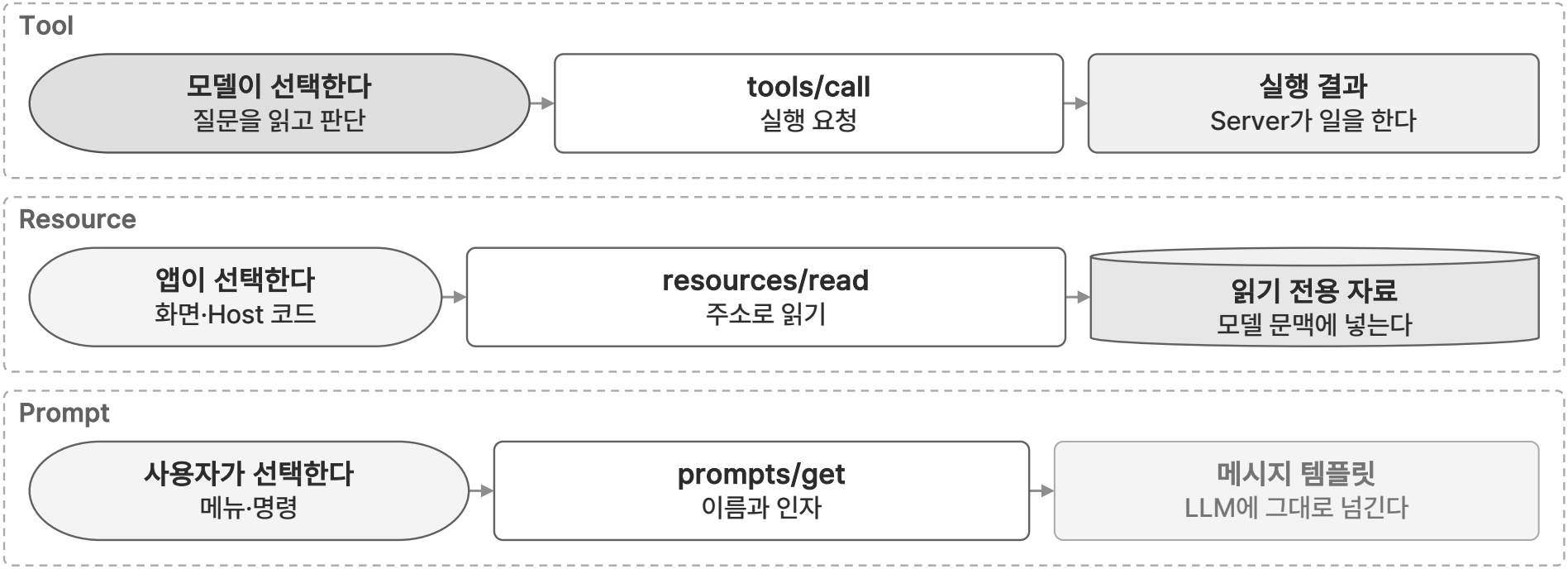


그림 3-1 세 가지 Primitive와 선택하는 주체 (Client-Server)

Tool/Resource/Prompt 차이와 용도

표 3-1 Tool, Resource, Prompt 비교

구분	무엇인가	누가 선택하나	FastMCP에서	예
Tool	실행하는 기능. 인자를 받아 결과를 돌려준다	모델	@mcp.tool	add, get_price
Resource	읽기 전용 자료. 주소(URI)로 읽는다	앱(Client)	@mcp.resource("scheme://...")	data://config
Prompt	미리 만들어 둔 지시문 템플릿	사용자	@mcp.prompt	ask_about_topic

표 3-2 상황별로 선택하기

이런 일이면	이것으로 낸다	예
모델이 판단해 조회·계산·변경을 실행한다	Tool	주문 조회, 환불 요청, 가격 계산
모델 문맥에 미리 넣을 읽기 전용 자료가 있다	Resource	설정값, 규정 원문, 사용자 정보
주소 하나로 비슷한 자료 여러 개를 읽는다	Resource 템플릿	repos://{owner}/{repo}/info
같은 형식의 요청을 반복한다	Prompt	리포트 작성, 코드 작성 요청, 역할 지정

Resource·Prompt를 구성하는 요소

Resource는 주소로, Prompt는 이름과 인자로 요청한다. 요청하는 함수와 돌려받는 형식이 요소마다 다르다.

표 3-3 Resource·Prompt 요소의 목적과 용도

요소	목적	언제 사용하나	예제에서
고정 Resource	주소 하나에 자료 하나를 둔다	설정값, 규정 원문처럼 늘 같은 자료	data://config
Resource 템플릿	주소 안의 변수로 비슷한 자료 여러 개를 읽는다	사용자별, 저장소별 정보	repos://{owner}/{repo}/info
주소 목록 조회	읽을 수 있는 주소 목록을 받는다	화면이나 Host가 선택할 자료를 보여 줄 때	list_resources(), list_resource_templates()
read_resource()	주소로 내용을 읽는다. 결과는 목록이다	모델 문맥에 원문을 넣을 때	contents[0].text
@mcp.prompt	지시문을 메시지로 조립한다. str, list[Message], PromptResult 중 하나를 돌려준다	같은 형식의 요청을 반복할 때	ask_about_topic, roleplay_scenario
get_prompt()	이름과 인자로 조립된 메시지를 받는다	사용자가 메뉴나 명령으로 선택할 때	get_prompt(이름, 인자 dict)

Resource와 Prompt를 LLM에 연결하는 방법

모델은 Tool만 호출할 수 있다. Resource와 Prompt를 모델이 사용하게 하려면 Host 코드가 가상 도구를 만들어 연결한다.

표 3-4 Host가 놓는 다리

구분	Resource	Prompt
모델에게 전달하는 정보	읽을 수 있는 주소 목록 (system 메시지)	프롬프트 이름과 필수 인자 (system 메시지)
모델에게 주는 가상 도구	read_resource(uri)	get_prompt(name, args)
모델이 선택하면	client.read_resource(uri)로 읽어 도구 결과로 돌려준다	client.get_prompt(name, args)로 받은 메시지를 대화에 추가한다

> **Capstone에서는**
> rag-mcp에 문서 원문을 읽는 Resource를 1개 이상 둔다. 검색 도구는 chunk를 돌려주고, 근거 패널은 Resource로 원문을 읽는다. Prompt는 선택 사항이다.

Resource로 자료 제공하기

@mcp.resource에 주소를 주면 함수의 반환값이 그 주소의 자료가 된다. 주소에 {owner} 같은 변수를 두면 템플릿이 되고, 변수 값이 함수 인자로 전달된다.

예제 3-1 Resource Server와 Client (발췌, FastMCP 4.1)

```
# Server: 읽기 전용 자료를 주소(URI)로 제공한다. 주소는 <scheme>://<path> 형식이면 된다
# 함수의 반환값이 그 주소의 내용이 된다
@mcp.resource("resource://greeting")              # 1) 고정 주소 + 문자열
def get_greeting() -> str:
    return "Hello from FastMCP Resources!"

@mcp.resource("data://config")                    # 2) 고정 주소 + JSON (dict → JSON 문자열)
def get_config() -> dict:
    return {"theme": "dark", "version": "1.2.0", "features": ["tools", "resources"]}

# 3) 템플릿 주소. {owner}, {repo} 변수 값이 같은 이름의 함수 인자로 전달된다
@mcp.resource("repos://{owner}/{repo}/info")
def get_repo_info(owner: str, repo: str) -> dict:
    return {"owner": owner, "name": repo, "full_name": f"{owner}/{repo}", "stars": 120}

# Client: 목록을 보고 주소로 읽는다
async with client:
    resources = await client.list_resources()                 # 고정 주소 목록
    templates = await client.list_resource_templates()        # 템플릿 목록 (.uri_template)
    contents = await client.read_resource("data://config")   # resources/read 요청
    config = json.loads(contents[0].text)                     # 결과는 목록. 문자열은 [0].text
    repo = await client.read_resource("repos://openai/gpt-4/info")   # 변수를 채운 주소
```

Prompt로 지시문 템플릿 만들기

@mcp.prompt 함수는 메시지를 조립해 str, list[Message], PromptResult 중 하나로 돌려준다.

예제 3-2 Prompt Server (발췌, FastMCP 4.1)

```
from fastmcp.prompts import Message, PromptResult

# Prompt는 미리 만들어 둔 지시문 템플릿이다. 함수 인자가 템플릿 변수가 된다
@mcp.prompt                                          # 1) 문자열 → user 메시지 하나
def ask_about_topic(topic: str) -> str:
    return f"'{topic}' 개념을 쉽게 설명해 주세요."

@mcp.prompt                                          # 2) PromptResult: 메시지 + 설명
def generate_code_request(language: str, task_description: str) -> PromptResult:
    text = f"{language}로 다음 작업을 수행하는 함수를 작성해 주세요: {task_description}"
    return PromptResult(messages=[Message(text)], description="코드 작성 요청")

@mcp.prompt                                          # 3) 메시지 여러 개. role로 assistant도 넣는다
def roleplay_scenario(character: str, situation: str) -> list[Message]:
    first = Message(f"역할극입니다. 당신은 {character}이고 상황은 {situation}입니다.")
    second = Message("알겠습니다. 준비가 완료되었습니다. 다음으로 뭘 할까요?", role="assistant")
    return [first, second]

# Client: 이름과 인자로 받는다 (prompts/get 요청)
# Server는 메시지를 조립만 하고, 일은 이 메시지를 받은 LLM이 한다
result = await client.get_prompt("ask_about_topic", {"topic": "MCP"})
for message in result.messages:
    print(message.role, message.content.text)    # user 'MCP' 개념을 쉽게 설명해 주세요.
```

Resource·Prompt 실행해 보기

전체코드 폴더의 resources, prompts로 각각 이동해 실행한다. LLM 없이 실행된다.

표 3-5 Resource·Prompt 실행 순서

순서	할 일 (명령)	보여야 하는 결과	다르면 확인할 것
1	python client_resources.py	resources: ['resource://greeting', 'data://config'], templates: ['repos://{owner}/{repo}/info']	고정 주소와 템플릿 주소가 따로 나오는지 본다
2	2번 출력의 나머지 줄	greeting: Hello from FastMCP Resources!, config.version: 1.2.0, stars: 120	read_resource 결과는 목록이라 [0].text로 추출한다
3	python client_prompts.py	prompts on server: 이름 네 개, [user] 'MCP' 개념을 쉽게 설명해 주세요.	Prompt는 메시지만 돌려준다. 모델 호출은 없다

Resource와 Prompt Summary

표 3-6 Resource와 Prompt 요점

구분	요점
세 기능	Tool은 모델이, Resource는 앱(화면·Host 코드)이, Prompt는 사용자가 선택한다
Resource	읽기 전용 자료를 URI로 제공한다. 주소에 {변수}를 두면 템플릿이 된다
Prompt	지시문 템플릿을 str, list[Message], PromptResult 중 하나로 돌려준다
LLM 연결	모델은 Tool만 호출할 수 있으므로 Host가 read_resource, get_prompt 같은 가상 도구로 연결한다
Capstone	rag-mcp에 문서 원문을 읽는 Resource를 1개 이상 둔다. Prompt는 선택 사항이다



4. MCP 통신 방식

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

전송 방식별 용도

1~3단계의 Client는 server.py를 하위 프로세스로 실행해 stdio로 연결했다. 팀 프로젝트처럼 Server를 먼저 실행해 두고 Agent 서버 여러 개가 함께 사용하려면 HTTP 전송으로 연결한다.

표 4-1 전송·연결 방식의 목적과 용도

방식	목적	언제 사용하나	주의
stdio	Client가 Server를 하위 프로세스로 실행하고 표준 입출력으로 메시지를 주고받는다	로컬 개발, 데스크톱 앱	Server는 부모의 환경 변수를 물려받지 않는다. stdout에 print하면 메시지가 깨진다
Streamable HTTP	실행 중인 Server에 POST /mcp로 메시지를 보낸다	compose 서비스, 원격 Server, 여러 Agent가 함께 쓰는 Server	주소 끝 /mcp를 빠뜨리면 404
HTTP+SSE	이전 명세의 HTTP 전송	이전 방식으로 만든 Server에 연결할 때	새 Server는 Streamable HTTP로 만든다
인메모리	Server 객체를 그대로 연결한다	단위 테스트	프로세스와 네트워크 없이 연결된다
Proxy	stdio Server처럼 보이면서 요청을 원격 HTTP Server로 넘긴다	stdio만 연결할 수 있는 프로그램이 원격 Server를 사용할 때	FastMCP 4.1은 create_proxy()

두 가지 전송 방식

MCP 메시지는 어느 방식이든 같은 JSON-RPC이고, 방식에 따라 메시지를 전달하는 경로만 달라진다. Server의 업무 코드는 그대로 둔다.

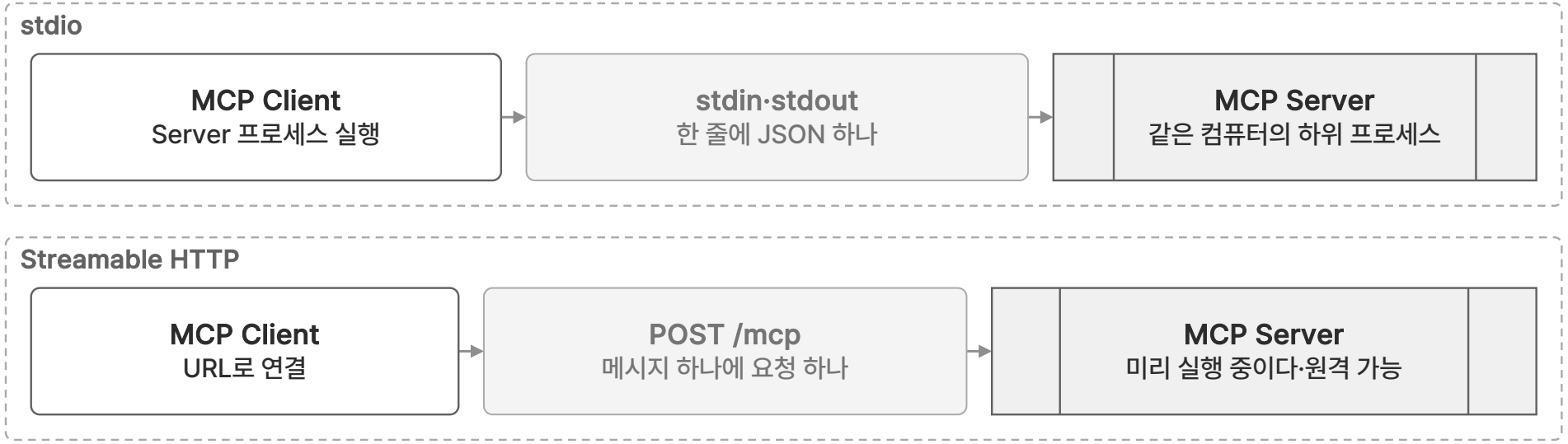


그림 4-1 stdio와 Streamable HTTP (Client-Server)

> **알아두기**
> MCP 전송 방식에는 stdio, Streamable HTTP 외에 HTTP+SSE도 있다. HTTP+SSE는 이전 명세의 방식이므로 새 Server는 Streamable HTTP로 만든다.

Capstone에서의 연결 방식

같은 Server를 상황에 따라 다른 방식으로 연결하고, compose로 실행할 때는 Streamable HTTP가 기본이다.

표 4-2 상황별 연결 방식

상황	방식	주소·설정
도구 단위 테스트	인메모리	Client(mcp). 테스트 코드에서 Server를 실행하지 않는다
로컬 개발·디버깅	stdio	Client(Path(".../server.py")) 또는 python -m
docker compose 실행	Streamable HTTP	http://business-mcp:9101/mcp, http://rag-mcp:9102/mcp
두 Server를 한 Client로	설정 (mcpServers)	도구 이름에 business_, rag_ 접두어가 붙는다



그림 4-2 Agent 서버의 연결 순서 (Pipe-and-Filter)

Server를 실행하는 두 가지 방법

stdio와 HTTP는 같은 Server를 실행하고, mcp.run()의 인자만 다르다.

표 4-3 stdio와 Streamable HTTP 비교

구분	stdio	Streamable HTTP
누가 실행하나	Client가 Server 프로세스를 실행한다	미리 실행 중이다 (compose 서비스, 원격 서버)
주소	파일 경로, 또는 command와 args	http://host:port/mcp (경로 /mcp 필수)
Server 코드	mcp.run()	mcp.run(transport="http", host=..., port=...)
사용처	로컬 개발, 데스크톱 앱(Claude Desktop, Cursor)	팀 프로젝트 기본. Agent 서버 여러 개가 같은 Server를 사용한다

Streamable HTTP로 주고받은 요청

원격 Server와 Client 사이에 중계를 두고 실제 요청을 기록했다. JSON-RPC 메시지 하나가 POST /mcp 요청 하나이고, 헤더에 버전과 메서드가 함께 실린다.

예제 4-1 원격 Server 실행 중 HTTP 기록 (FastMCP 4.1, 본문 일부 생략)

```
POST /mcp HTTP/1.1
accept: application/json, text/event-stream     # 두 형식을 모두 받겠다고 알린다
content-type: application/json
mcp-protocol-version: 2026-07-28
mcp-method: server/discover                     # 본문을 열지 않아도 무슨 요청인지 보인다
{"jsonrpc":"2.0","id":1,"method":"server/discover","params":{"_meta":{...}}}

HTTP/1.1 200 OK
content-type: application/json
{"jsonrpc":"2.0","id":1,"result":{"supportedVersions":["2026-07-28"],"capabilities":{...}}}

POST /mcp HTTP/1.1
mcp-method: tools/call
mcp-name: multiply                              # 호출하는 도구 이름도 헤더에 실린다
{"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"name":"multiply","arguments":{"a":3,"b":7}}}

HTTP/1.1 200 OK
{"jsonrpc":"2.0","id":3,"result":{"content":[{"type":"text","text":"21.0"}],
 "structuredContent":{"result":21.0},"isError":false}}
```

Client로 연결하는 네 가지 방법

Client(...)에 무엇을 넘기느냐만 다르고, 연결한 뒤의 동작은 같다.

표 4-4 Client에 넘기는 것

방식	Client(...)에 넘기는 것	사용처	주의
① 인메모리	Server 객체 mcp	단위 테스트	프로세스 없이 바로 연결한다
② stdio	Path("rag_mcp/server.py"), 또는 command와 args	로컬 개발	상대 import가 있는 패키지는 python -m으로
③ HTTP	"http://127.0.0.1:9101/mcp"	팀 프로젝트 기본	경로 /mcp를 빠뜨리지 않는다
④ 설정	{"mcpServers": {...}}	여러 Server를 한 Client로	도구 이름 앞에 '서버이름_' 접두어가 붙는다

Client로 연결하는 네 가지 방법 (계속)

예제 4-2 connect.py (발췌)

```
# 같은 Server에 연결하는 네 가지 방법. Client(…)에 넘기는 값만 다르다
# ① 인메모리: Server 객체를 그대로 넘긴다. 프로세스·네트워크 없이 테스트에 사용한다
client1 = Client(mcp)
# ② stdio: .py 경로. Client가 Server 프로세스를 실행하고 표준 입출력으로 연결한다
client2 = Client(Path("rag_mcp/server.py"))
# ③ Streamable HTTP: URL. Server가 먼저 실행 중이어야 하고, 주소 끝은 /mcp이다
client3 = Client("http://127.0.0.1:9101/mcp")
# ④ 설정(dict): 여러 Server를 Client 하나로 연결한다
#    도구 이름 앞에 Server 이름이 붙는다 (business_get_order)
client4 = Client({"mcpServers": {
    "business": {"url": "http://127.0.0.1:9101/mcp", "transport": "http"},   # HTTP Server
    "rag": {"command": sys.executable, "args": ["-m", "rag_mcp.server"]},   # stdio Server
}})
```

원격 Server와 Proxy

도구를 HTTP로 실행하면 다른 컴퓨터에서도 연결할 수 있다. stdio Server만 연결할 수 있는 프로그램에는 Proxy를 두어 원격 Server를 로컬처럼 보이게 한다.

예제 4-3 원격 Server와 Proxy (발췌, FastMCP 4.1)

```
# 원격 Server: 같은 도구를 HTTP로 실행한다
if __name__ == "__main__":
    mcp.run(
        transport="http",          # Streamable HTTP (FastMCP 2의 "streamable-http"와 같다)
        host="0.0.0.0",            # 다른 컴퓨터에서도 연결할 수 있다. 방화벽에서 포트를 연다
        port=8000,
        path="/mcp",               # Client 주소: http://서버주소:8000/mcp
    )

# Client: 파일 경로 대신 URL을 넘긴다. Server는 미리 실행 중이어야 한다
async with Client("http://서버주소:8000/mcp") as client:
    result = await client.call_tool("multiply", {"a": 3, "b": 7})

# Proxy: 원격 HTTP Server를 로컬 stdio Server처럼 보이게 한다
from fastmcp.server import create_proxy          # FastMCP 2의 FastMCP.as_proxy 를 대신한다

proxy = create_proxy("http://서버주소:8000/mcp", name="calculator Proxy")
proxy.run()                        # stdio로 실행된다

# 주의: stdio Server는 부모의 환경 변수를 물려받지 않는다. 설정은 env로 직접 넘긴다
PythonStdioTransport("proxy_server.py", env={"REMOTE_MCP_URL": "http://서버주소:8000/mcp"})
```

연결 오류와 해결

오류 메시지는 예제로 재현해 확인했다.

표 4-5 자주 발생하는 연결 오류

증상	원인	해결
Client failed to connect	Server를 실행하지 않았거나 포트가 다르다	Server를 먼저 실행하고 주소·포트를 확인한다
MCPError: Not Found (404)	URL에 /mcp를 빠뜨렸다	http://host:port/mcp
Unknown tool: 'get_order'	여러 Server 설정에서는 이름에 접두어가 붙는다	business_get_order
stdio에서 Connection closed	상대 import가 있는 Server를 파일 경로로 실행했다	command와 args로 python -m 실행
Proxy의 도구 목록이 빈 목록	stdio Server가 환경 변수를 물려받지 못해 다른 주소로 연결했다	PythonStdioTransport(..., env={...})로 넘긴다
DeprecationWarning (uriTemplate 등)	mcp 2.x에서 필드 이름이 snake_case로 바뀌었다	uri_template, input_schema, destructive_hint

원격 Server를 두 터미널로 실행하기

Streamable HTTP는 Server를 먼저 실행해 두고 Client가 주소로 연결한다. 터미널 두 개를 열어 하나는 Server, 하나는 Client로 사용한다. 두 터미널 모두 전체코드 폴더에서 가상환경을 켜고 remote 폴더로 이동한 뒤 실행한다.

표 4-6 원격 Server 실행 순서

순서	할 일 (명령)	보여야 하는 결과	다르면 확인할 것
1	터미널 1: python server_remote.py	FastMCP 배너가 나오고 입력을 기다린다	터미널 1은 닫지 않는다. 종료는 Ctrl+C
2	터미널 2: python client_remote.py	Client connected: True, Available tools: ['multiply'], multiply result: 21.0	Client failed to connect면 터미널 1의 Server가 실행 중인지 본다
3	터미널 1을 Ctrl+C로 종료한 뒤 터미널 2에서 다시 실행	연결 오류가 난다	Server를 먼저 실행해야 한다는 것을 확인하는 단계이다
4	터미널 2에서 주소를 지정해 실행: REMOTE_MCP_URL=http://127.0.0.1:8000/mcp python client_remote.py	1번과 같은 결과	주소 끝 /mcp를 빠뜨리면 404

MCP 통신 방식 Summary

표 4-7 MCP 통신 방식 요점

구분	요점
전송 방식	stdio(로컬, 하위 프로세스)와 Streamable HTTP(원격, POST /mcp). 메시지는 같은 JSON-RPC이다
Server 실행	같은 Server를 두 방식으로 실행하며, mcp.run()의 인자만 다르다
Client 연결	인메모리, stdio(.py 경로), HTTP(URL), 설정(mcpServers) 네 가지 방법이 있다
Capstone	compose로 실행할 때는 Streamable HTTP가 기본이다
연결 오류	Server 실행 여부, 주소·포트, 주소 끝의 /mcp를 먼저 확인한다



5. 레퍼런스 아키텍처

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

아키텍처 설계 기준

팀 프로젝트는 MCP Server와 LLM Agent를 여러 서비스로 나눠 배포한다. 서비스를 나누고 연결하는 방식은 아래 다섯 가지 요구 사항에서 확인한다.

표 5-1 요구 사항과 아키텍처 결정

요구 사항	아키텍처 결정	확인하는 도식
LLM 응답은 수 초에서 수십 초가 걸린다	질문은 작업으로 접수해 202를 바로 돌려주고, 진행 상황은 SSE 이벤트로 보낸다	동기 경로와 비동기 경로, 처리 순서
업무 규칙은 한 곳에서만 관리한다	business-mcp는 DB에 직접 접근하지 않고 Spring Boot 내부 API(/internal/**)를 호출한다	구성도 5번, MCP 연결과 문서 색인
브라우저에 노출되는 주소를 줄인다	공개 포트는 화면과 Spring Boot뿐이고, 나머지 서비스는 docker network 안에 둔다	배포 아키텍처
데이터를 바꾸는 도구는 사람이 확인한다	변경 도구 앞에서 Agent가 상태를 저장하고 멈춘 뒤, 승인을 받으면 같은 thread_id로 재개한다	전체 흐름 (2)
답변에는 근거 문서가 있어야 한다	문서는 질문과 별개로 미리 색인하고, rag-mcp가 Vector DB에서 근거를 찾는다	MCP 연결과 문서 색인

아키텍처 뷰 구성

그림 한 장으로 모든 질문에 답하지 않는다. 구성도에서 전체를 본 뒤, 뷰마다 질문 하나를 그 질문에 맞는 스타일로 그린다.

표 5-2 뷰와 스타일

뷰	답하는 질문	스타일	도식
전체 구성도	어떤 서비스가 있고 무엇과 연결되는가	Container 관점, Client-Server	레퍼런스 아키텍처 구성도
논리 뷰	책임을 어떤 계층으로 나누고 누가 누구를 호출하는가	Layered	논리 아키텍처의 계층 구조
런타임 뷰	메시지가 어떤 순서로 오가는가	Sequence, Event-Driven	동기·비동기 경로, 처리 순서, 전체 흐름, 진행 이벤트
배포 뷰	무엇이 어느 tier와 경계 안에서 실행되는가	N-tier, 배포 관점	배포 아키텍처
연결 뷰	도구와 근거 문서는 어디에서 오는가	Client-Server, Pipe-and-Filter	MCP 연결과 문서 색인
Harness	모델을 감싼 코드는 무엇으로 구성되는가	Microkernel	Harness의 여덟 구성 요소

도식 표기 읽는 법

모양과 선마다 뜻이 정해져 있다. 상자 모양으로 저장소, 외부 서비스, 배포 노드를 구분하고, 화살촉으로 동기 호출과 비동기 메시지를 구분한다.

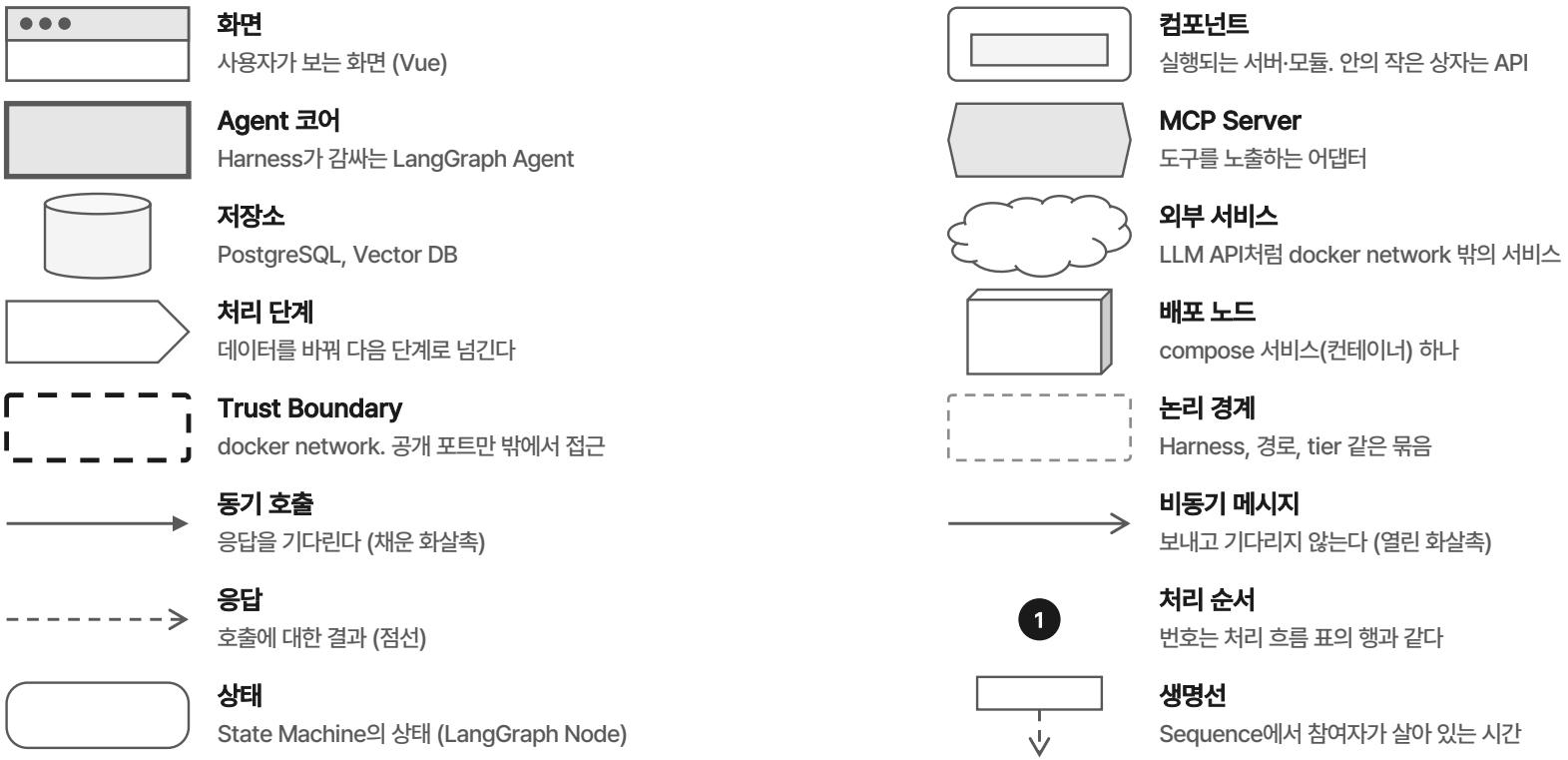


그림 5-1 도식 표기 (범례)

레퍼런스 아키텍처 구성도

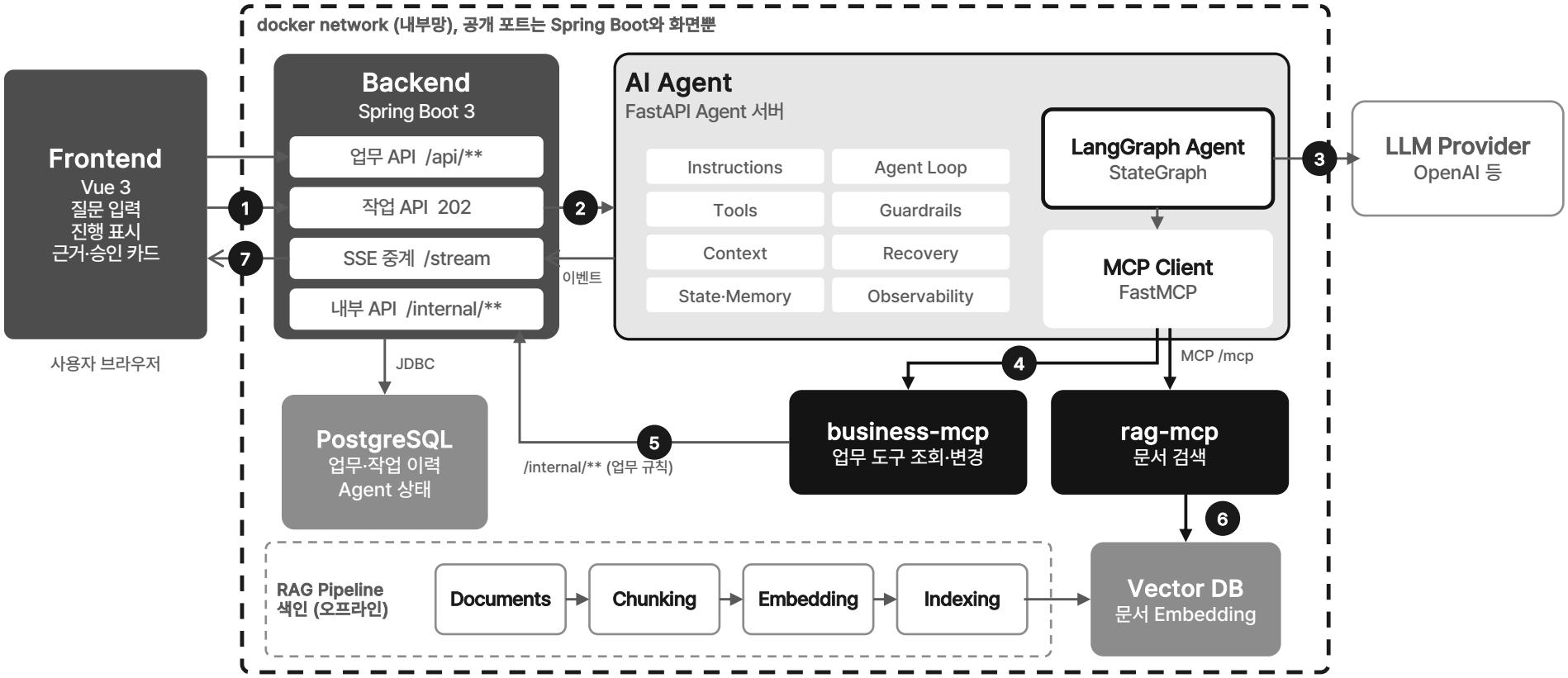


그림 5-2 레퍼런스 아키텍처 구성도 (Container 관점, Client-Server)

구성도의 처리 흐름

구성도의 번호를 따라 읽으면 질문 하나가 화면에서 출발해 Agent 서버와 MCP Server, 저장소를 거쳐 다시 화면으로 돌아온다.

표 5-3 번호별 처리

번호	구간	처리 내용	프로토콜
1	Vue → Spring Boot 작업 API	POST /api/ai/tasks. 질문으로 작업을 만들고 202 { taskId }를 바로 받는다	HTTPS
2	Spring Boot → Agent 서버	작업 상태를 저장한 뒤 Agent 서버에 작업 실행을 요청한다	내부 HTTP
3	LangGraph Agent → LLM API	Harness가 Instructions·Context를 붙여 모델을 호출하고, 모델이 다음 도구를 제안한다	HTTPS
4	MCP Client → MCP Server	제안된 도구를 tools/call로 호출한다. 변경 도구는 Guardrails가 승인 전까지 멈춘다	MCP (/mcp)
5	business-mcp → Spring Boot	업무 데이터는 /internal/**로 Spring Boot를 거쳐 읽고 바꾼다	내부 HTTP
6	rag-mcp → Vector DB	질문과 가까운 문서 chunk를 찾아 근거로 반환한다	검색 질의
7	Agent 서버 → Spring Boot → Vue	진행 이벤트(TOKEN·TOOL_START·APPROVAL_REQUIRED·DONE)를 SSE로 중계한다	SSE

구성 요소별 목적과 책임

구성 요소마다 따로 두는 이유와 역할, 금지 사항을 정한다. 평가에서는 금지 사항을 어긴 코드가 없는지 본다.

표 5-4 목적, 역할, 금지 사항

구성 요소	따로 두는 이유	역할	금지 사항
Vue 3	화면과 업무 처리를 나눈다	질문 입력, 진행 상태·근거·승인 화면	권한 판단, AI 서버 직접 호출
Spring Boot	업무 규칙과 데이터 변경을 한 곳에 모은다	업무 API, 작업 생성·상태 저장, SSE 중계, 내부 API	LLM 호출
FastAPI Agent	오래 걸리는 LLM 호출을 업무 서버와 분리한다	Harness로 모델을 감싸고 도구 선택, 승인 대기	DB 직접 쓰기
business-mcp	업무 기능을 Agent가 호출하는 도구로 노출한다	조회·변경 도구, /internal/** 호출	업무 규칙 복제
rag-mcp	문서 검색을 도구로 분리해 근거를 반환한다	Vector DB 검색, 출처 반환	업무 데이터 변경
PostgreSQL	업무 데이터와 Agent 상태를 저장한다	업무 데이터, 작업 이력, checkpointer	해당 없음
Vector DB	문서 chunk를 Embedding으로 검색한다	문서 chunk, Embedding, 출처 메타데이터	해당 없음

기술 스택

계층마다 사용하는 기술과 기준 버전이다.

표 5-5 계층별 기술

계층	기술	기준 버전	역할
Presentation	Vue 3 (Vite)	3.x	질문 입력, 진행 상태·근거·승인 화면
Application	Spring Boot	3.x	업무 API, 작업 접수와 상태 저장, SSE 중계
AI Serving	FastAPI, LangGraph, LangChain	LangGraph 1.x, LangChain 1.x	Agent 실행, 승인 대기와 재개
Tool	FastMCP (Server·Client)	FastMCP 4.1, mcp 2.x	업무 기능과 문서 검색을 도구로 제공하고 호출한다
Knowledge	RAG Pipeline, Vector DB	Chroma 또는 pgvector	문서 색인과 근거 검색
Data	PostgreSQL	16	업무 데이터, 작업 이력, Agent 상태 (checkpointer)
External	LLM API	OpenAI 등	모델 호출. 키는 .env에만 둔다

논리 아키텍처의 계층 구조

책임을 다섯 계층으로 나누고, 위 계층은 바로 아래 계층만 호출한다.

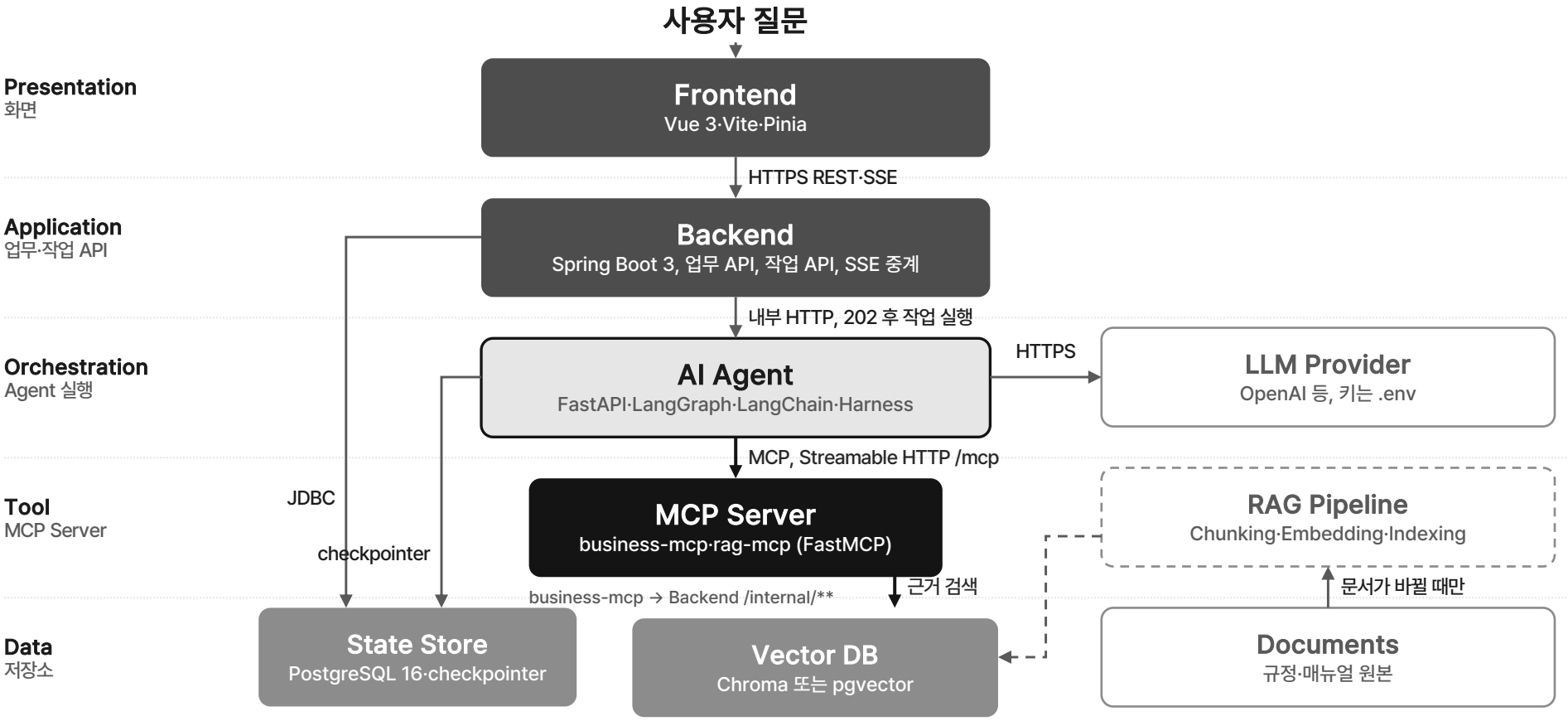


그림 5-3 논리 아키텍처 (Layered, 가운데 열이 요청 경로)

Layer별 책임과 인터페이스

layer는 코드의 책임 구분이다. 위 layer는 바로 아래 layer의 인터페이스만 사용하고, layer마다 금지 사항이 정해져 있다.

표 5-6 Layer별 책임

Layer	구성 요소	역할	금지 사항	아래 layer와의 인터페이스
Presentation	Vue 3	질문 입력, 진행 상태·근거·승인 화면	권한 판단, AI 서버 직접 호출	REST /api/**, SSE 구독
Application	Spring Boot	업무 API, 작업 생성·상태 저장, SSE 중계	LLM 호출	내부 HTTP로 Agent 호출, PostgreSQL 접근
AI Serving	FastAPI Agent (LangGraph·Harness)	의도 분류, 도구 선택, RAG, 승인 대기	DB 직접 쓰기	MCP (Streamable HTTP /mcp), LLM API
Tool	business-mcp, rag-mcp	업무 API와 문서 검색을 도구로 노출	업무 규칙 복제	business-mcp는 /internal/**, rag-mcp는 Vector DB
Data	PostgreSQL, Vector DB	업무 데이터, 작업 이력, Agent 상태, 문서 Embedding	해당 없음	없음 (가장 아래 layer)

> **위 layer를 호출하는 예외**
>
> business-mcp가 Application layer의 /internal/**를 호출하는 것은 아래에서 위로 가는 호출이다. 업무 규칙을 Spring Boot 한 곳에 두려는 의도된 예외이고, /internal/**는 docker network 안에서만 열린다.

동기 경로와 비동기 경로

요청을 걸리는 시간으로 나눠, 업무 요청은 바로 답하고 LLM 요청은 작업으로 접수한다.

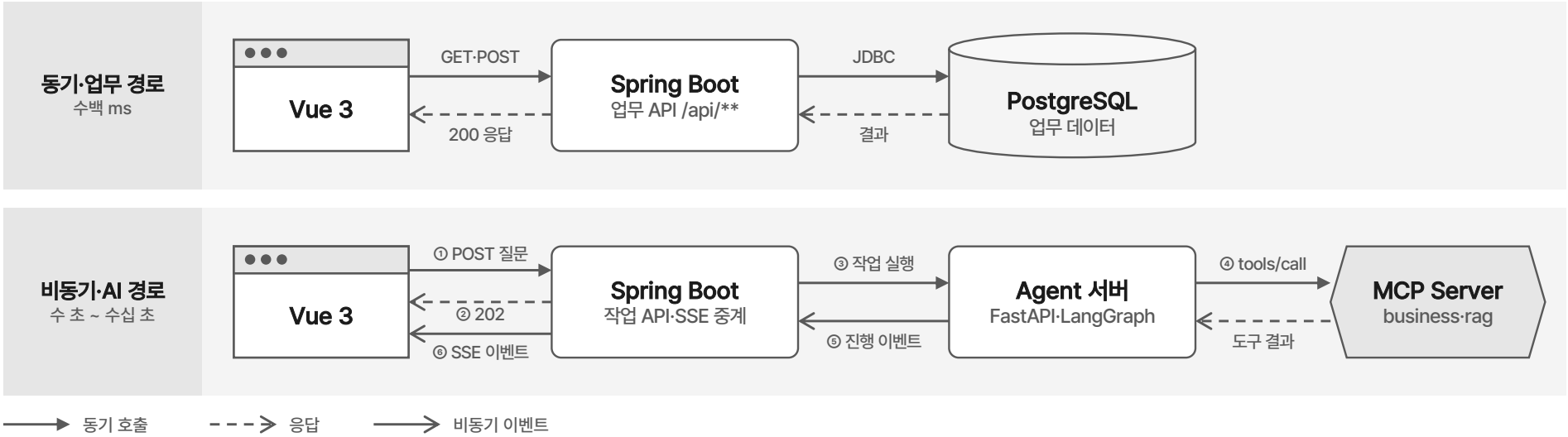


그림 5-4 두 경로 (Request-Reply와 비동기 작업)

표 5-7 경로별로 거치는 일

경로	거치는 일	응답
동기	업무 조회·등록, 작업 상태 조회, 승인·거절 기록	바로 답한다
비동기	Agent 질의응답, 문서 색인·재색인, 일괄 분석·리포트, 승인 후 재개	202 후 이벤트(SSE)로

질문 하나가 처리되는 순서

Spring Boot는 202를 돌려준 뒤 요청을 끝내고, 결과는 이벤트로 다시 받는다.

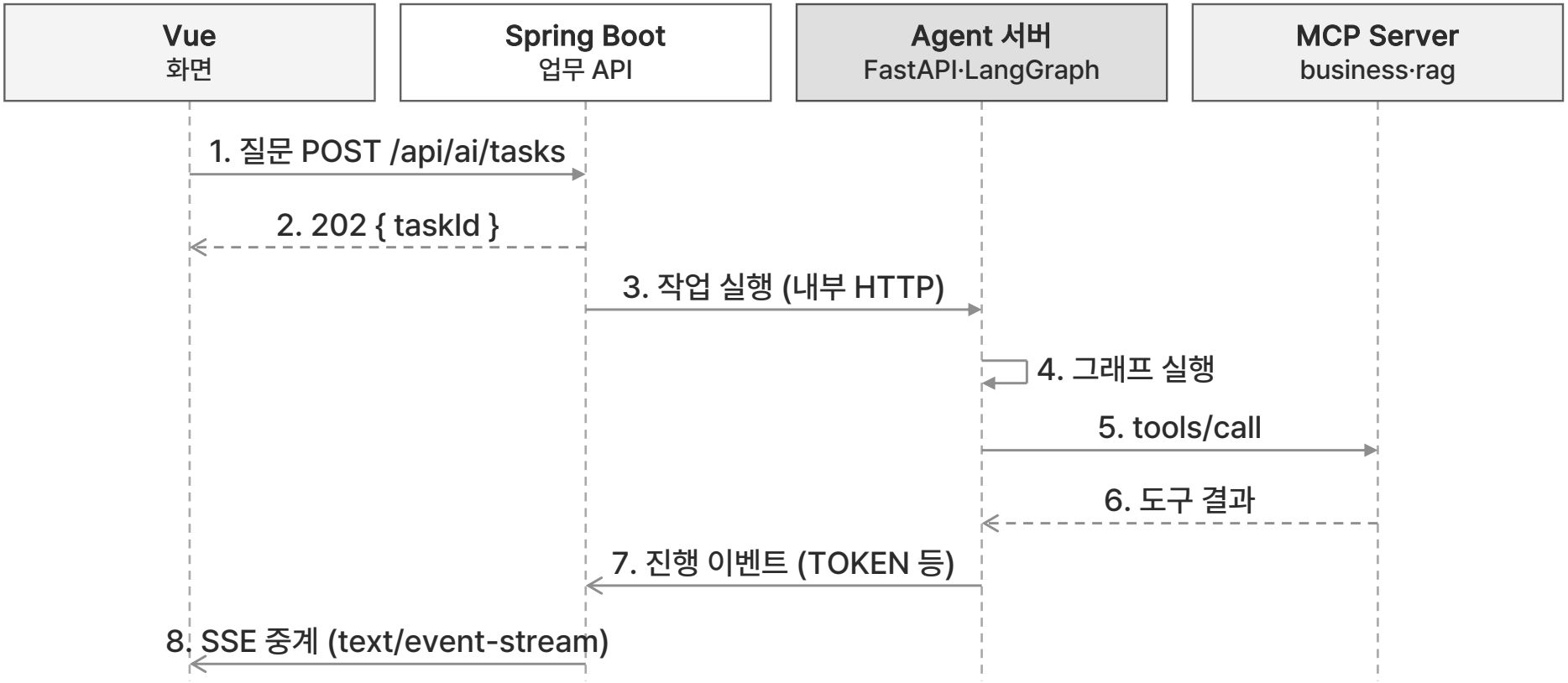


그림 5-5 처리 순서 (Sequence)

전체 흐름 (1) 질문 접수와 근거 검색

질문 하나가 화면에서 Vector DB까지 내려갔다가 진행 이벤트로 돌아오는 과정이다.

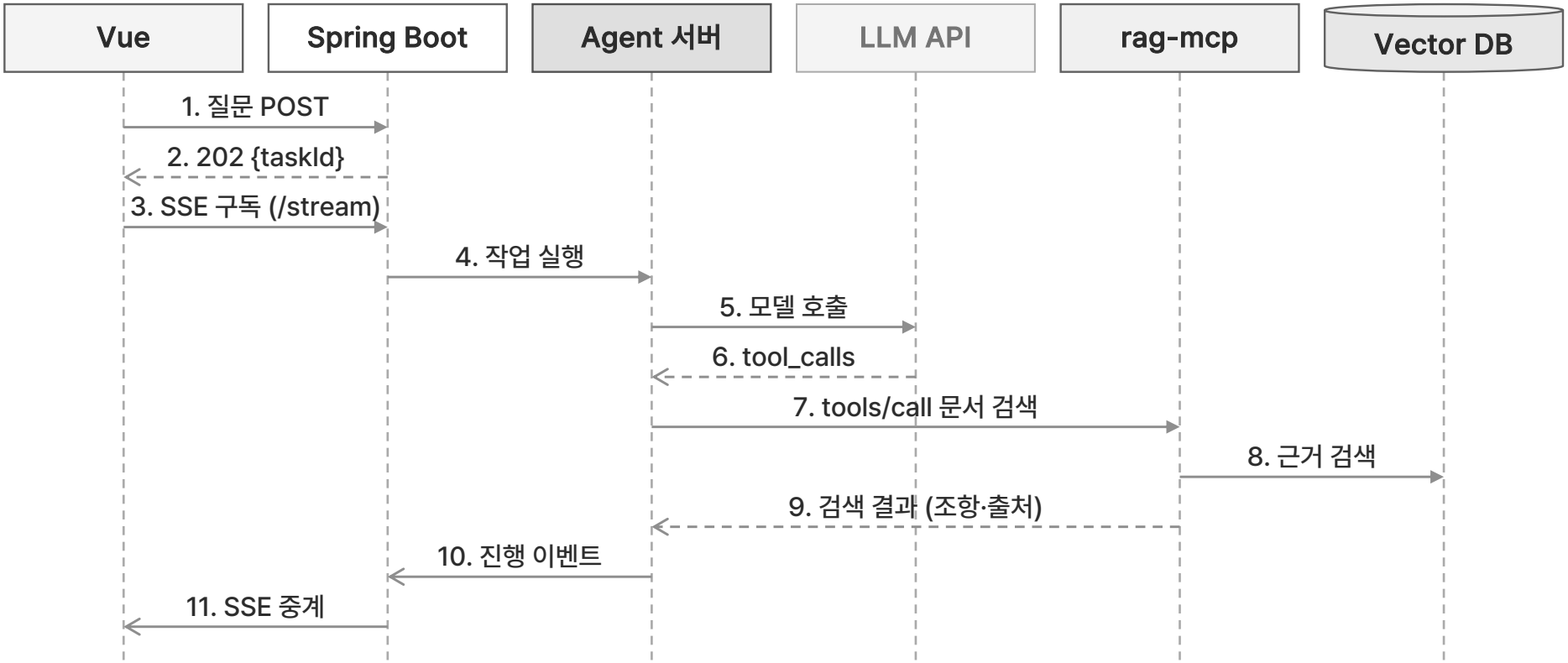


그림 5-6 질문 접수와 근거 검색 (Sequence)

전체 흐름 (2) 변경 도구 승인과 완료

변경 도구를 선택하면 Agent는 상태를 저장하고 멈추고, 승인 뒤에는 Spring Boot가 데이터를 바꾼다.

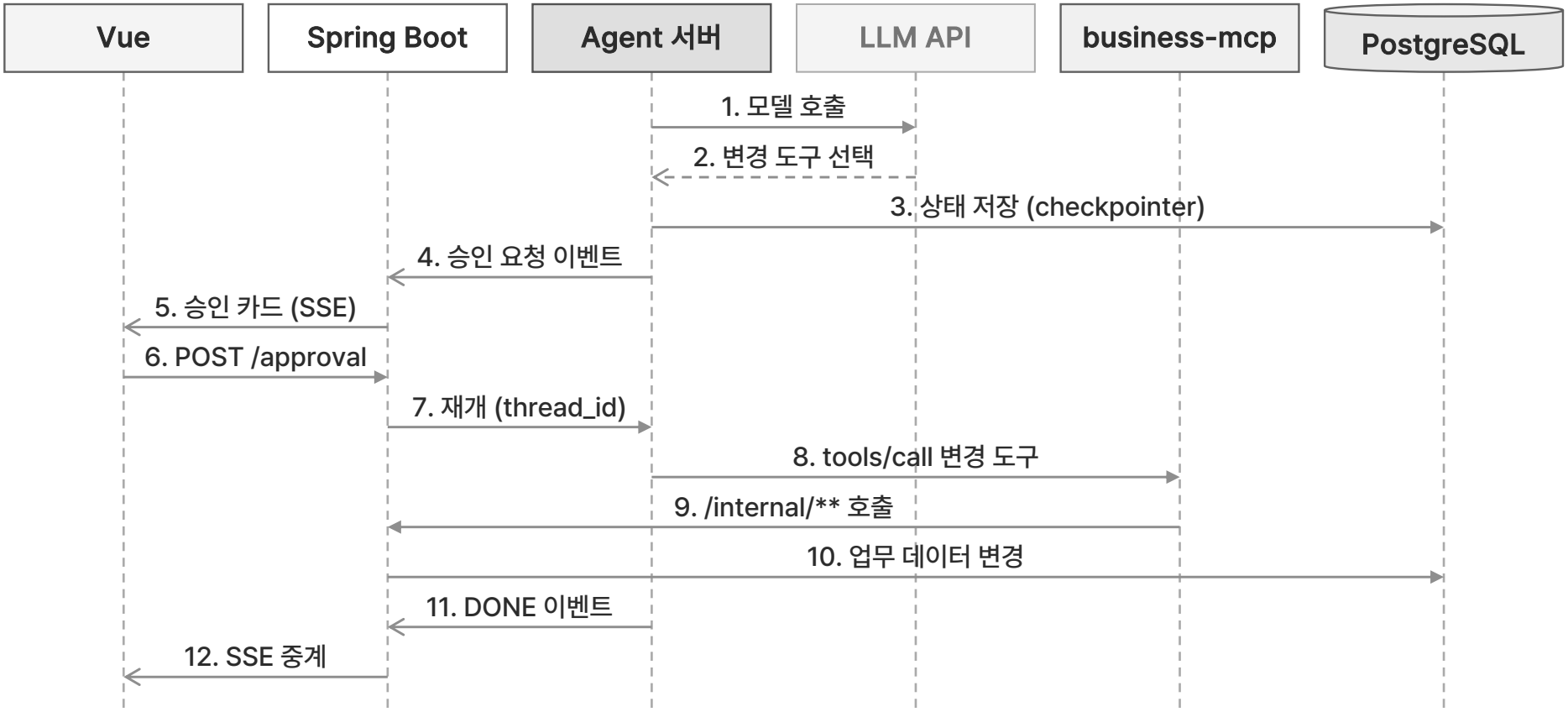


그림 5-7 변경 도구 승인과 완료 (Sequence)

API 계약

Vue는 다섯 API만 호출하고, AI 서버 주소는 브라우저에 드러내지 않는다.

표 5-8 화면이 호출하는 API

Method	Path	설명	응답
POST	/api/ai/tasks	질문으로 작업을 만든다	202 { taskId }
GET	/api/ai/tasks/{id}	상태와 결과를 조회한다	{ status, answer, sources }
GET	/api/ai/tasks/{id}/stream	진행 이벤트를 받는다	text/event-stream
POST	/api/ai/tasks/{id}/approval	승인·거절을 보낸다	202
GET	/api/ai/tasks?mine=true	내 작업 이력	목록

표 5-9 화면이 받는 이벤트

이벤트	언제	화면 표현
TOKEN	답변이 생성되는 동안	답변 영역에 이어서 표시한다
TOOL_START·TOOL_END	MCP 도구 실행 전후	'주문 조회 중…' 상태 표시
APPROVAL_REQUIRED	변경 도구 앞에서 멈췄을 때	승인 카드 (도구·인자·금액)
DONE·FAILED	작업이 끝났을 때	스트림을 닫고 작업 목록 갱신

진행 이벤트의 발행과 구독

Agent 서버는 진행 상황을 이벤트로 발행하고, Spring Boot가 SSE로 중계하면 화면의 각 영역이 필요한 이벤트만 받아 표시한다. Agent 서버는 어느 화면 영역이 이벤트를 받는지 알지 못한다.

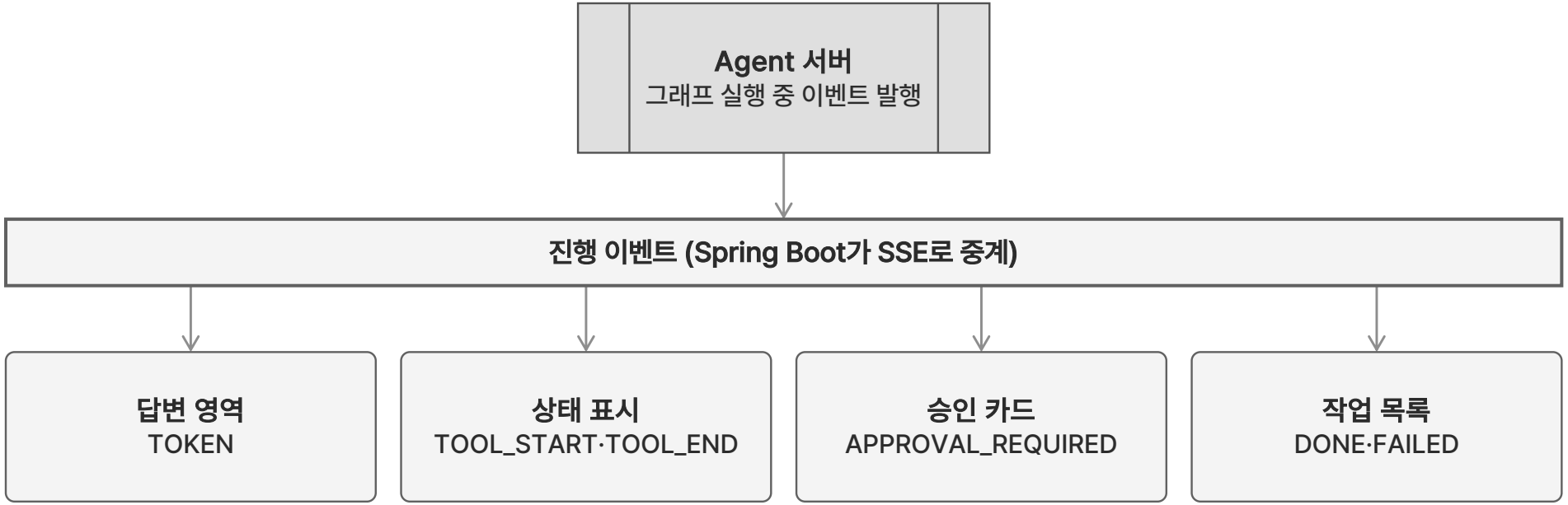


그림 5-8 Agent 진행 이벤트 (Event-Driven, Publish-Subscribe)

> **알아두기**
> 새 이벤트를 받는 화면 영역을 추가해도 Agent 서버 코드는 바뀌지 않는다. 이벤트 이름과 형식은 API 계약에 적은 대로 화면과 Spring Boot가 같이 따른다.

배포 아키텍처

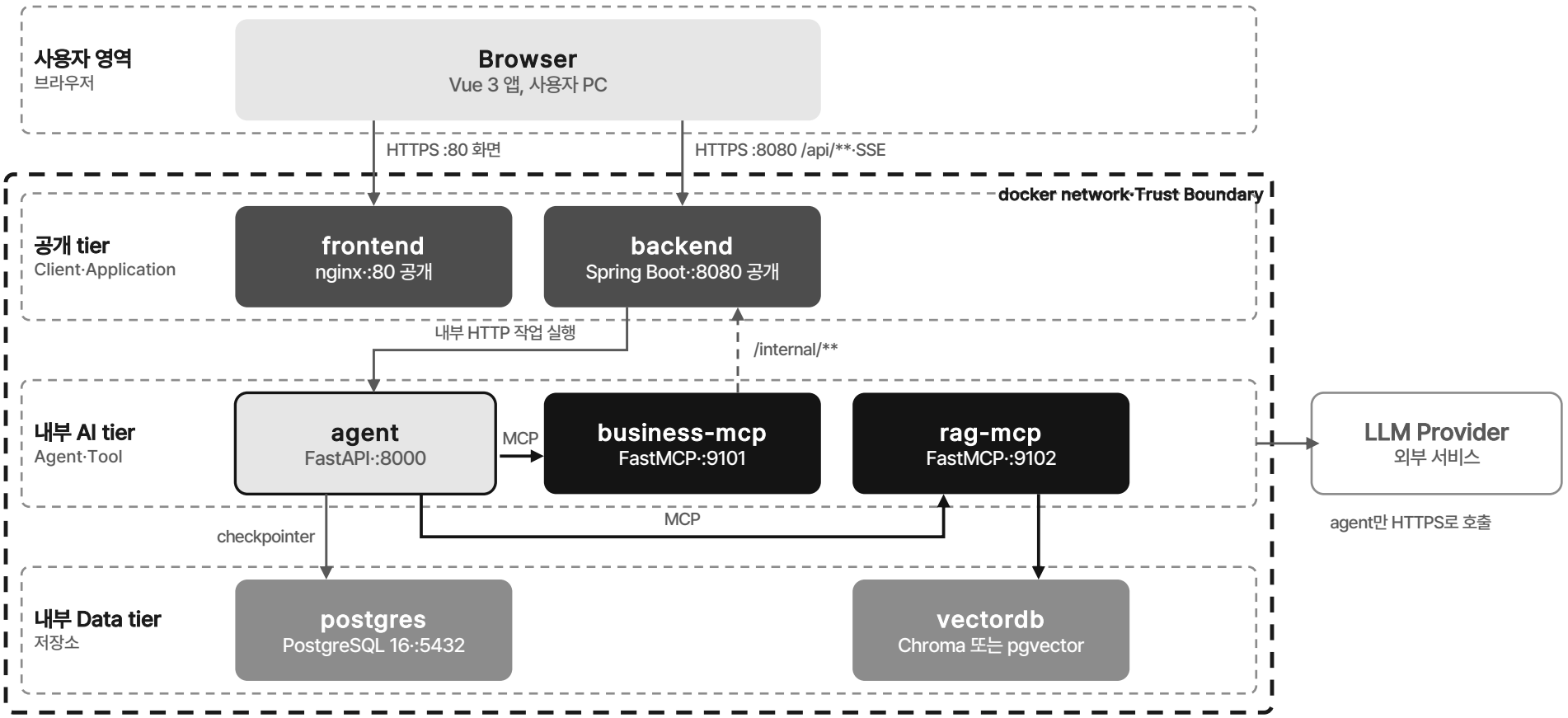


그림 5-9 배포 아키텍처 (N-tier, 배포 관점과 Trust Boundary)

배포 구성 (docker compose)

서비스 이름과 포트는 예시이고, compose 안에서는 서비스 이름을 주소로 사용한다.

표 5-10 서비스와 기동 순서

서비스	기술	포트 (예시)	공개	먼저 실행 중이어야 하는 것
frontend	Vue 3 (nginx)	80	공개	backend
backend	Spring Boot	8080	공개	postgres, agent
agent	FastAPI Agent 서버	8000	내부	business-mcp, rag-mcp
business-mcp	FastMCP	9101	내부	backend (/internal/**)
rag-mcp	FastMCP	9102	내부	vectordb
postgres	PostgreSQL 16	5432	내부	없음
vectordb	Chroma 또는 pgvector	8001 또는 5432	내부	없음

Layer별 Tier 배치 구조

layer는 논리적인 역할 분리, tier는 물리적·배포 관점의 분리를 의미함.
한 tier에 여러 layer가 들어갈 수 있으므로, 어느 layer가 어느 tier의 어느 컨테이너에서 실행되는지 표로 맞추어 본다.

표 5-11 Layer와 Tier

Layer (책임)	Tier (실행 위치)	컨테이너 (compose 서비스)	포트·공개	접근하는 쪽
Presentation	Client tier	frontend (Vue 3, nginx)	80·공개	브라우저
Application	Application tier	backend (Spring Boot)	8080·공개	frontend, business-mcp (/internal/**)
AI Serving	AI tier	agent (FastAPI)	8000·내부	backend
Tool	AI tier	business-mcp, rag-mcp (FastMCP)	9101, 9102·내부	agent
Knowledge	Data tier	vectordb (Chroma 또는 pgvector)	8001 또는 5432, 내부	rag-mcp
Data	Data tier	postgres (PostgreSQL 16)	5432·내부	backend, agent (checkpointer)

MCP 연결과 문서 색인

Agent는 MCP로 두 Server의 도구를 호출하고, 근거 문서는 질문과 별개로 미리 색인해 둔다. business-mcp는 DB 대신 Spring Boot 내부 API를 호출한다.

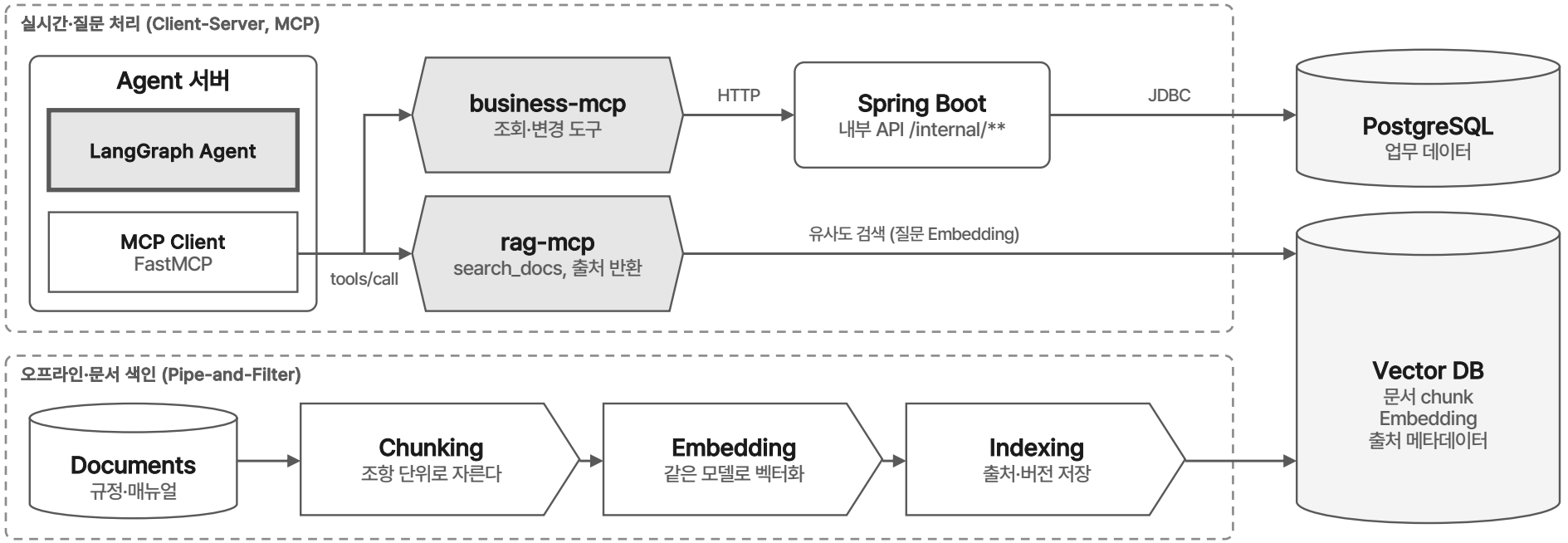


그림 5-10 MCP 연결과 색인 경로 (실시간 Client-Server, 색인 Pipe-and-Filter)

> **알아두기**
> Embedding 모델 이름과 차원을 색인 메타데이터에 남긴다. 검색할 때 질문도 같은 모델로 Embedding해야 가까운 문서를 찾는다.

Capstone 레퍼런스 아키텍처 Summary

표 5-12 Capstone 레퍼런스 아키텍처 요점

구분	요점
설계 기준	응답 시간, 업무 규칙, 노출 범위, 승인, 근거 다섯 가지 요구 사항에서 구조를 정했다
구성도	Vue, Spring Boot, FastAPI Agent, MCP Server 둘, 저장소 둘과 외부 LLM API로 구성된다
두 경로	업무 경로는 Spring Boot에서 끝나고, LLM을 호출하는 AI 경로는 202 접수 뒤 SSE 이벤트로 결과를 받는다
Trust Boundary	공개 포트는 frontend와 backend뿐이고, 나머지는 docker network 안에 둔다
MCP 연결	business-mcp는 DB 대신 Spring Boot 내부 API를 호출하고, rag-mcp는 미리 색인한 Vector DB를 검색한다



6. Harness

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

Agent = Model + Harness

레퍼런스 아키텍처의 Agent 서버 안에서 모델의 호출을 통제하는 코드가 Harness이다. 모델은 다음 응답이나 호출할 도구를 제안만 하고, 실행, 한도, 기록, 복구는 모델을 감싼 Harness가 담당한다.

표 6-1 Model과 Harness의 역할 분담

Model (LLM)	Harness (모델을 감싸는 코드)
다음 응답이나 호출할 도구를 제안한다	프롬프트와 도구를 선택해 모델에 넘긴다
도구를 직접 실행하지 못한다	제안된 도구를 실행하고 결과를 돌려준다
몇 번째 시도인지, 비용이 얼마인지 모른다	반복 횟수, 시간, 권한을 제한한다
대화가 끝나면 아무것도 기억하지 않는다	상태를 저장하고, 실패하면 복구한다
같은 모델이면 팀마다 차이가 없다	모든 단계를 기록한다. 팀마다 이 부분이 다르다

Harness 동작 확인 방법

관측과 평가 두 가지로 확인하고, 둘 다 제품소개자료의 증빙 화면이 된다.

표 6-2 관측과 평가

관측 (Observability)	평가 (Evaluation)
LangSmith로 trace 수집	프롬프트, 한도, 모델을 바꾸면 평가셋을 다시 실행
메시지, 로그, trace에 같은 trace_id	바꾸기 전과 후의 결과표를 나란히 남긴다
Node·도구·Token·지연·비용 기록	한도 초과, 재시도, 대체 모델 발생 건수
FAILED 작업은 trace 링크를 작업 이력에 저장	장애 시연을 같은 스크립트로 반복

Harness의 여덟 구성 요소

왼쪽 넷은 모델에 들어가는 입력을, 오른쪽 넷은 실행을 통제하고 기록한다.



그림 6-1 Harness 구성 (Microkernel)

구성 요소별 구현 기준

각 요소의 적용 위치와 최소 기준이다. 값은 권장값이고, 팀이 바꾸면 Harness 명세에 남긴다.

표 6-3 Harness 구현 기준

구성 요소	적용 위치	최소 기준
Instructions	prompts/*.md	역할, 도구 사용 규칙, 금지 사항, 출력 형식. 코드에 직접 넣지 않고 버전 관리
Tools	MCP Client → 도구 변환	도구 결과는 길이를 잘라 넘기고, 오류는 ToolMessage로 돌려준다
Agent Loop	LangGraph StateGraph	최대 8턴, 작업 120초, 같은 도구 3회까지
Context	메시지 정리·RAG 주입	Token 예산을 정하고, 넘으면 오래된 대화를 요약한다
State·Memory	checkpointer·store	thread_id 단위로 저장하고, 승인 대기 후 그대로 재개
Guardrails	approval Node 또는 HITL middleware	변경 도구는 interrupt로 멈춘다. 도구 결과의 개인정보는 마스킹 처리
Recovery	모델 호출·Agent 서버	429, 503은 2회 재시도, 계속 실패하면 대체 모델 또는 FAILED
Observability	LangSmith	trace_id 하나로 Node, 도구, Token, 지연 여부를 모니터링

Harness Summary

표 6-13 Harness 요점

구분	요점
정의	Agent = Model + Harness. 모델은 다음 응답이나 호출할 도구를 제안만 하고, 실행·한도·기록·복구는 Harness가 담당한다
8개 구성 요소	Instructions, Tools, Context, State·Memory, Agent Loop, Guardrails, Recovery, Observability
구현 방식	B는 create_agent + middleware, A는 직접 만든 StateGraph이다
승인	변경 도구는 destructiveHint로 표시하고, 실행 전에 멈춰 checkpointer에 상태를 저장한 뒤 사람의 결정을 기다린다
확인	관측(trace)과 평가로 확인하고, 요소마다 테스트를 하나씩 반영한다

팀별 Capstone 평가표

표 6-14 Capstone 평가 항목과 배점

항목	주요 내용	배점
Biz. 아이템	- 참신한 Biz. 아이템이면서 B2B나 B2B2C 대상 - 상용화되었을 때 효용성이 매우 크다고 판단되는 아이템은 팀 프로젝트에서 연속성 유지 가능 (팀 프로젝트 연속성 유지는 운영진과 교수진이 동의한 경우에만 가능)	10
아키텍처 설계	- Biz. 아이템을 위한 전체 AI 아키텍처 설계 - ISO/IEC 25010(구 ISO/IEC-9126) 품질평가를 고려한 아키텍처 설계	10
AI Agent 설계	- State 구조와 Graph 흐름(Node, 분기, 반복)이 업무에 맞게 설계됨 - Agentic AI 요소가 설계됨	15
AI Agent 구현	- 설계한 Graph가 코드에 반영되고, 모델이 도구를 선택해 근거와 함께 답함 - 상태 저장·재개와 진행 이벤트가 동작함 - Agentic AI 요소가 구현됨	15
MCP 설계·구현	- 업무에 필요한 MCP 설계·구현 - business-mcp, rag-mcp가 조회·변경 도구와 설명을 갖추고 Agent가 연결해 호출함 - 오류 처리와 예외 처리가 적절히 동작함	20
Harness	- Harness의 주요 구성 요소가 적절하게 적용됨	20
인증	- 사용자 인증 처리가 적용됨	10
가산점 (대기열)	- RabbitMQ, Kafka 같은 대기열 솔루션으로 작업을 큐에 넣어 처리하고, 실패한 작업을 재시도함	10
기본 합계		**100**
가산점 포함		**110**



수고하셨습니다.

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX